

PRD: Local Data-Driven Portfolio & Activity Log

1. Project Overview

Goal: Build a high-performance, SEO-optimized portfolio using Next.js that renders content from local data structures (JSON/Markdown).

Objective: To consolidate all internship activities into a single "Lego-style" codebase that demonstrates technical proficiency and SOLID principles.

2. Technical Stack

- **Framework:** Next.js (App Router).
 - **Language:** TypeScript (Strict mode).
 - **Styling:** Tailwind CSS.
 - **Data Handling:** Local **JSON** for projects and **MDX** or **Markdown** for logs/blogs.
 - **Deployment:** Vercel or Netlify.
-

3. Functional Requirements

3.1 Data Architecture

- **projects.json:** Must include Title, Description, Tech Stack (array), Completion Date, and Category (e.g., "SEO Audit," "Frontend," "Research").
- **Markdown Content:** Weekly learning logs or technical documentation stored in **/content** and parsed dynamically.
- **Type Safety:** All data must be mapped to specific TypeScript Interfaces to ensure no missing properties.

3.2 Core Pages & Navigation

- **Home (/):** Hero section + "Featured" highlights + Live Internship Timeline.
- **Work (/work):** A gallery of all activities with a **Filter System** (filter by tech stack or category).
- **Logs (/logs/[slug]):** Dynamic routes generated via **generateStaticParams** for reading weekly internship logs.

3.3 Component Architecture ("Lego-Style")

- **Atomic Design:** Break UI into **atoms** (Buttons, Badges) and **molecules** (ProjectCards, NavLinks).
- **Layouts:** Persistent navigation and footer with active-link styling.
- **Loading States:** Implementation of **loading.tsx** skeletons for transitions.

4. Non-Functional Requirements

4.1 Performance & SEO

- **Lighthouse Score: 90+** in Performance, Accessibility, Best Practices, and SEO.
- **Image Optimization:** Every image must use the Next.js `<Image/>` component with specific `width/height` or `fill` to prevent layout shifts.
- **Metadata API:** Implementation of dynamic `generateMetadata` for the log pages to ensure unique SEO titles.

4.2 Mobile Responsiveness

- **Fluid Grids:** Use Tailwind's grid/flex system to transition from 1 column (mobile) to 3 columns (desktop).
- **Touch-Ready:** Minimum 44x44px hit targets for all interactive buttons and links.
- **Performance:** No layout shifts (CLS) on mobile devices.

5. Project Milestones (Phasing)

Phase	Milestone	Deliverables
1	Data & Schema	Create <code>projects.json</code> and basic Markdown logs; define TS Interfaces.
2	Base Architecture	Set up Next.js App Router, Tailwind, and Global Layout.
3	Dynamic Routing	Implement file-system reading logic to generate <code>/logs/[slug]</code> pages.
4	UI/UX Development	Build the Filterable Work Gallery and Responsive Navigation.

5	Optimization	Audit via Lighthouse; compress images; add Meta Tags.
6	Deployment	Host on Vercel and submit the live link for final review.

6. Success Criteria

- New projects can be added simply by editing the **JSON** file without touching the UI code.
- The site is fully accessible (screen reader friendly).
- The code follows **SOLID** principles (e.g., the Filter logic is separate from the UI display).